LAPORAN JOBSHEET
“12- Incremental Static Regeneration (ISR)”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Implementasi ISR Otomatis

Bagian 1 – Tambahkan revalidate

1. Buka halaman static.tsx pada folder src/pages/product

```
// Fungsi getStaticProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil da
export const getStaticProps = async () => {
  const res = await fetch("http://localhost:3000/api/product");
  const response = await res.json();

  // console.log("Data produk yang diambil dari API:", response);

  return {
    props: {
      products: response.data, // Pastikan untuk memberikan nilai default jika data tidak ters
    },
    revalidate : 10, // Opsi ini akan membuat halaman di-regenerate setiap 10 detik
  };
};
```

Menambahkan revalidate

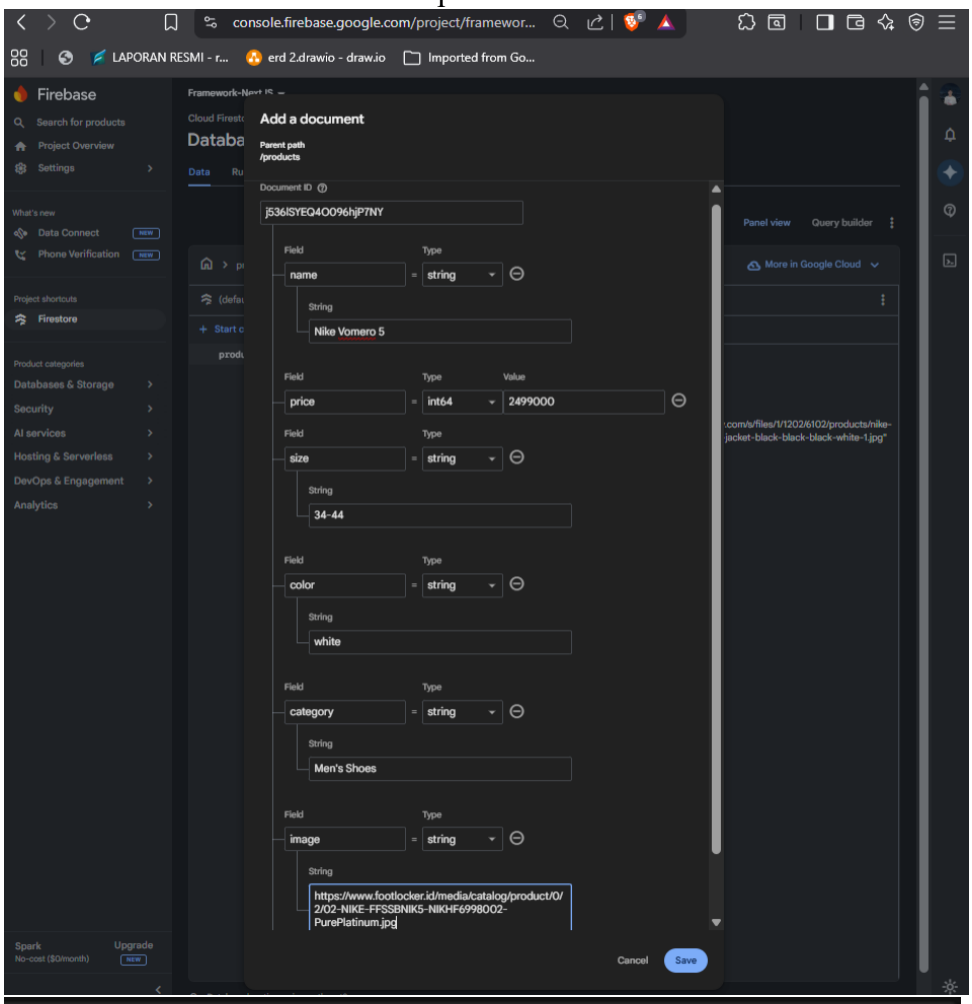
Bagian 2 – Pengujian ISR

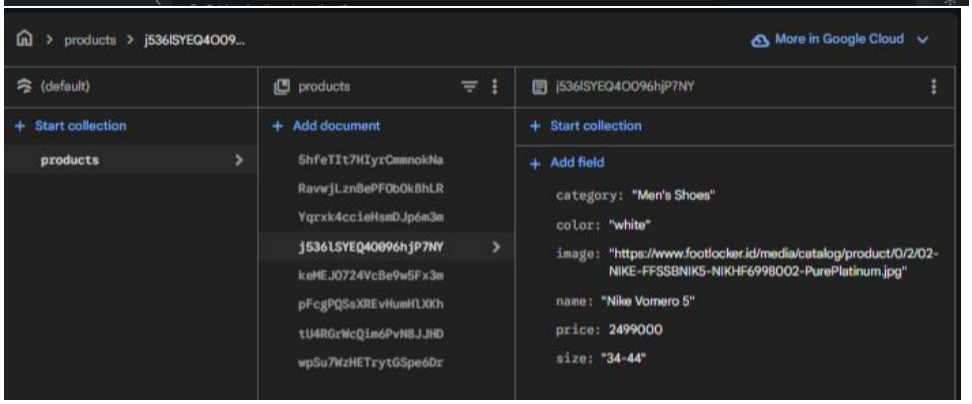
Jalankan: (lakukan hal sama seperti JS sebelumnya untuk ngebuild SSG)

- npm run build (jika berhasil)

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-rout
ing> npm run build
 ├   /_app
 ├ o /404
 ├ o /about
 ├ ƒ /api/[[...product]]
 ├ ƒ /api/hello
 ├ o /auth/login
 ├ o /auth/register
 ├ o /blog
 ├ o /blog/[slug]
 ├ o /category/[[...slug]]
 ├ o /product
 ├ ● /product/[product] (1149 ms)
 │ ├ /product/5hfeTIt7HIyrCmnokNa
 │ ├ /product/RavwjLznBePFObOkBhLR
 │ ├ /product/Yqrxk4ccieHsmDJp6m3m
 │ └ [+4 more paths]
 ├ ƒ /product/server
 ├ ● /product/static (434 ms)             10s      1y
 ├ o /profile
 ├ o /profile/edit
 ├ o /setting/app
 ├ o /shop/[[...slug]]
 ├ o /user
 └ o /user/password

o  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

- Tambahkan data baru di database pada firebase

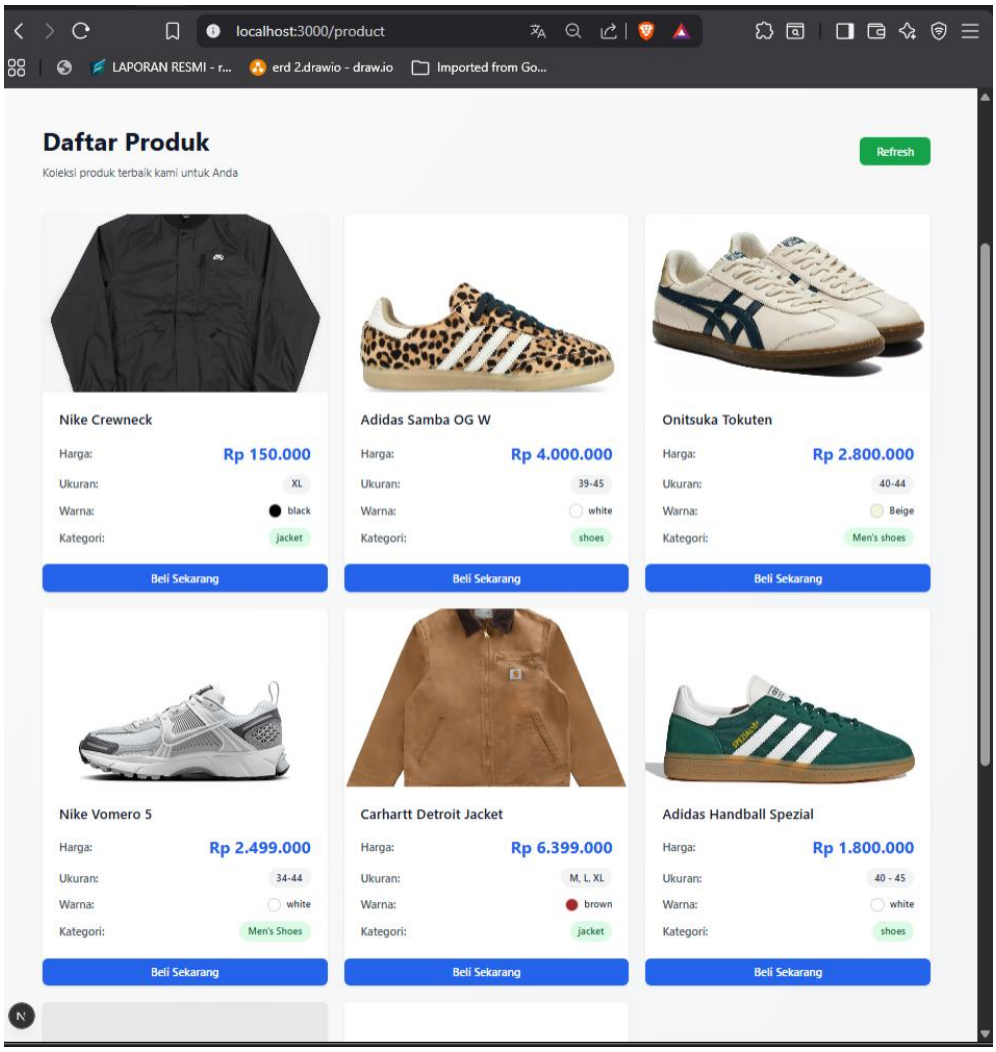




- Refresh halaman sebelum 10 detik → Data lama.
  Sebelum 10 detik data yang akan ditampilkan masih data lama

  Data terupdate otomatis setelah 10 detik setelah di refresh

Refresh setelah 10 detik → Data baru muncul.

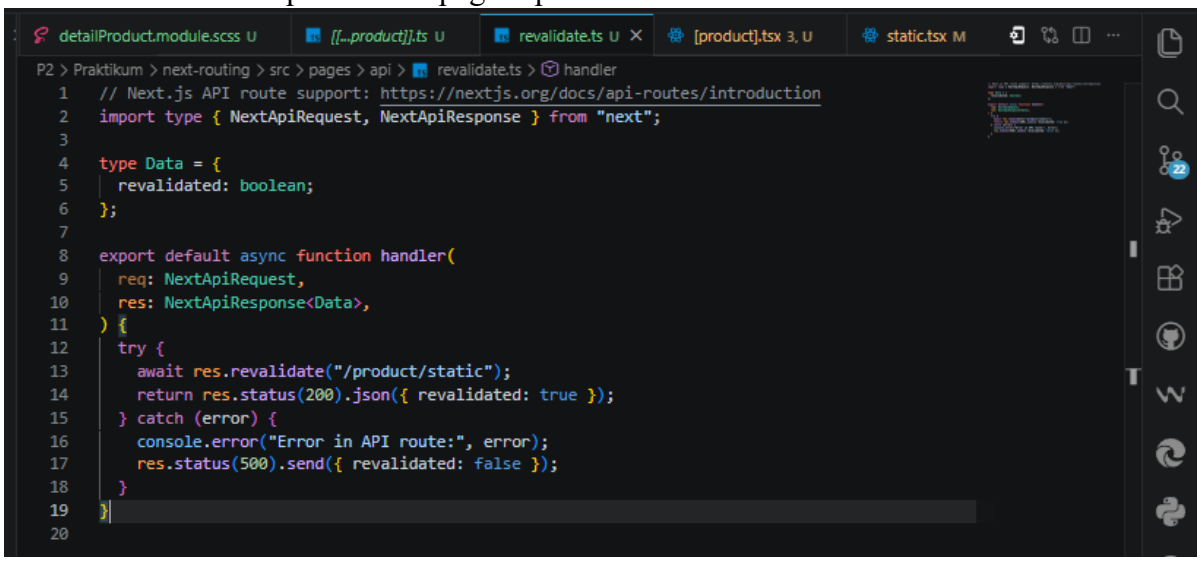


Data terupdate otomatis setelah 10 detik setelah di refresh

On-Demand Revalidatio

Bagian 1 – Buat API Revalidate

1. Buat file revalidate.ts pada folder pages/api/ dan modifikasi



Note: :jika kita menggunakan code ini ada sedikit masalah yaitu Ketika salah satu data dihapus maka data yang terhapus tidak secara otomatis terupdate pada layar meskipun sudah di refresh selama 10 detik. Untuk bisa terupdate data yang ditampilkan diperlukan trigger dengan mengakses apinya http://localhost:3000/api/product

Bagian 2 – Tambahkan Parameter Data

Untuk mengatasi hal tersebut (pada bagian 1) maka suatu kondisi pada file revalidat

- Modifikasi file revalidate.ts

```
P2 > Praktikum > next-routing > src > pages > api > revalidate.ts > handler > message
1  // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2  import type { NextApiRequest, NextApiResponse } from "next";
3
4  type Data = {
5    revalidated: boolean;
6    message?: string;
7  };
8
9  export default async function handler(
10   req: NextApiRequest,
11   res: NextApiResponse<Data>,
12 ) {
13   if (req.query.data === "product") {
14     try {
15       await res.revalidate("/product/static");
16       return res.status(200).json({ revalidated: true });
17     } catch (error) {
18       console.error("Error in API route:", error);
19       res.status(500).send({ revalidated: false });
20     }
21   }
22
23   return res.json({
24     revalidated: false,
25     message: "Invalid query parameter. Expected 'data=product'.",
26   });
27 }
28
```

- Uji coba menambahkan parameter dan value pada url http://localhost:3000/api/revalidate?data=product maka akan muncul true dan sesuai dengan kondisi (req.query.data ===”product”)

```
localhost:3000/api/revalidate?data=product
LAPORAN RESMI - r...   erd 2.drawio - draw.io   Imported from Go...
{
  "revalidated": true
}
```

- Uji coba dengan url http://localhost:3000/api/revalidate?data=

```
localhost:3000/api/revalidate?data=
LAPORAN RESMI - r...   erd 2.drawio - draw.io   Importe
{
  "revalidated": false,
  "message": "Insert correct token"
}
```

Bagian 3 – Tambahkan Token Security

Perlu ditambahkan paramater agar user tidak merubah data melalui url

- Buka file .env dan modifikasi

```
P2 > Praktikum > next-routing > .env.local
1  FIREBASE_API_KEY=AIzaSyDen7Mk5dq0CLAuXfcwnlvBLDZrmualWTQ
2  FIREBASE_AUTH_DOMAIN=framework-nextjs.firebaseapp.com
3  FIREBASE_PROJECT_ID=framework-nextjs
4  FIREBASE_STORAGE_BUCKET=framework-nextjs.firebasestorage.app
5  FIREBASE_MESSAGING_SENDER_ID=348929355692
6  FIREBASE_APP_ID=1:348929355692:web:ae9edc3db1add71bad5409
7
8  REVALIDATE_TOKEN=12345678
```

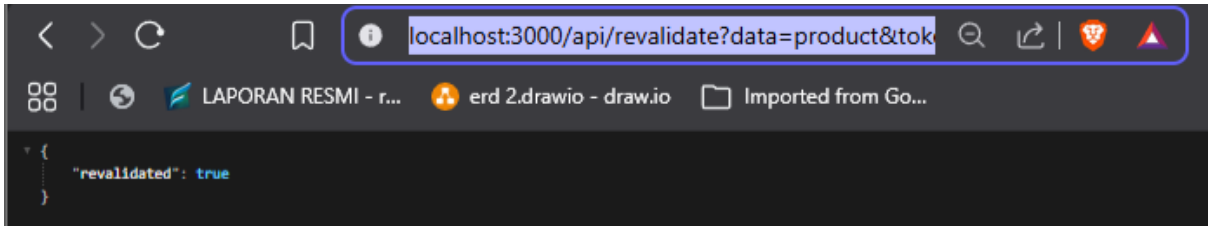
- Modifikasi file revalidate.ts tambahkan kondisi pada line 13 – 17

```
) {
  if (req.query.token !== process.env.REVALIDATE_TOKEN) {
    return res.status(401).json({
      revalidated: false,
      message: "Insert correct token",
    });
  }
```

Pengujian Manual Revalidation

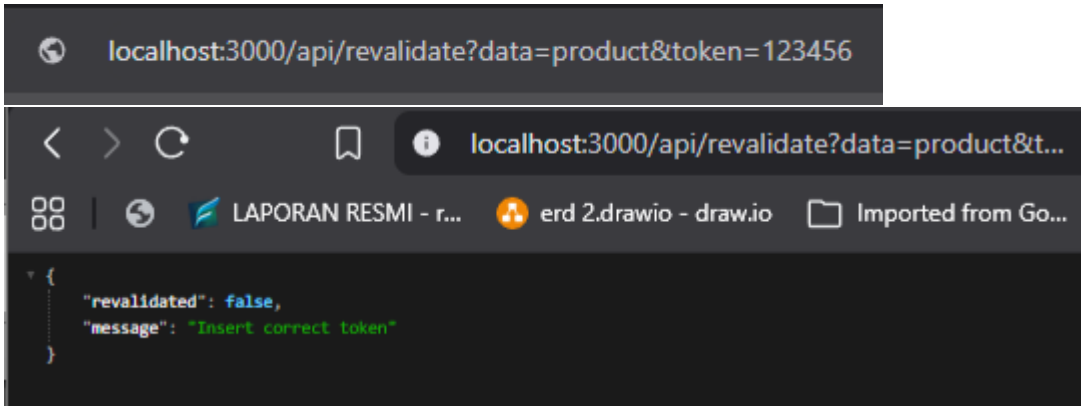
Jalankan http://localhost:3000/api/revalidate?data=product&token=12345678

Jika Benar



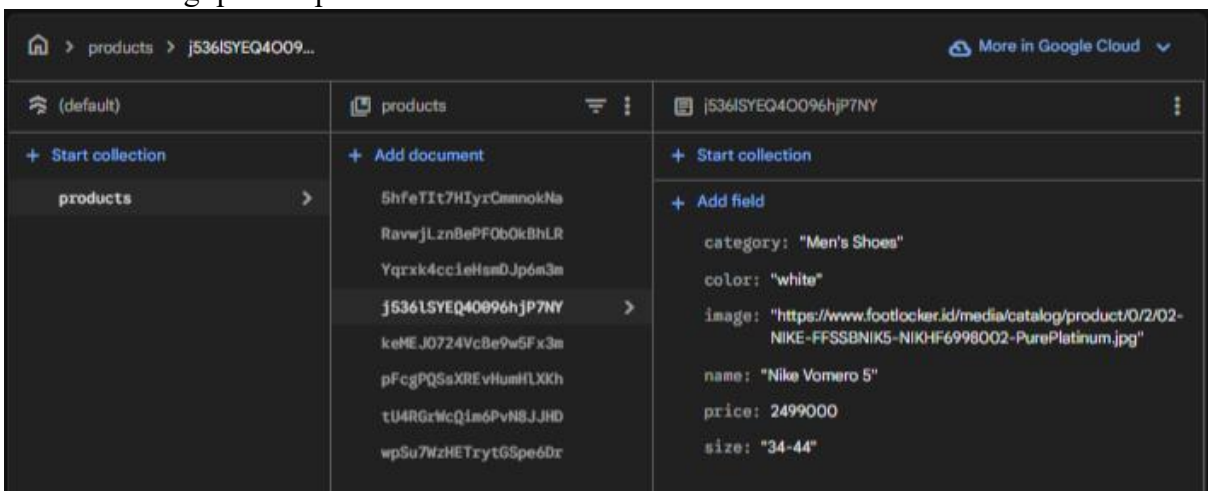
{ revalidate: true }

Jika Salah



TUGAS PRAKTIKUM

1. Tambahkan lagi produk pada firebase



2. Implementasikan ISR dengan revalidate: 10.

```
// Fungsi getStaticProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil da
export const getStaticProps = async () => {
  const res = await fetch("http://localhost:3000/api/product");
  const response = await res.json();

  // console.log("Data produk yang diambil dari API:", response);

  return {
    props: {
      products: response.data, // Pastikan untuk memberikan nilai default jika data tidak ters
    },
    revalidate : 10, // Opsi ini akan membuat halaman di-regenerate setiap 10 detik
  };
};
```

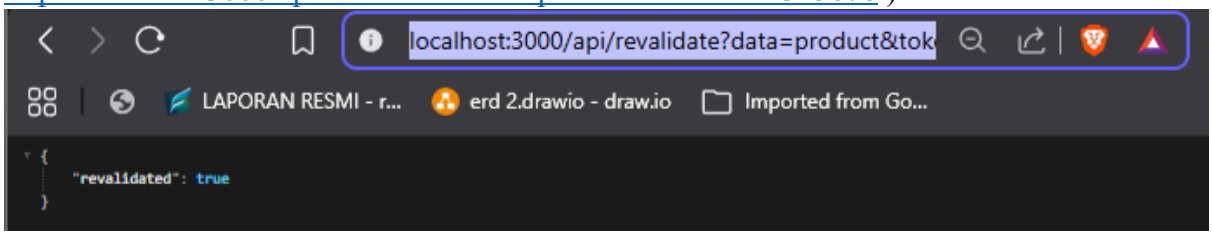
3. Tambahkan endpoint On-Demand Revalidation.

```
P2 > Praktikum > next-routing > src > pages > api > revalidate.ts > handler
// Next.js API route support: https://nextjs.org/docs/api-routes/introduction
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
  revalidated: boolean;
  message?: string;
};

export default async function handler(
  req: NextApiRequest,
  res: NextApiResponse<Data>,
) {
  if (req.query.token !== process.env.REVALIDATE_TOKEN) {
    return res.status(401).json({
      revalidated: false,
      message: "Insert correct token",
    });
  }

  if (req.query.data === "product") {
    try {
      await res.revalidate("/product/static");
      return res.status(200).json({ revalidated: true });
    } catch (error) {
      console.error("Error in API route:", error);
      res.status(500).send({ revalidated: false });
    }
  }

  return res.json({
    revalidated: false,
    message: "Invalid query parameter. Expected 'data=product'.",
  });
}
```

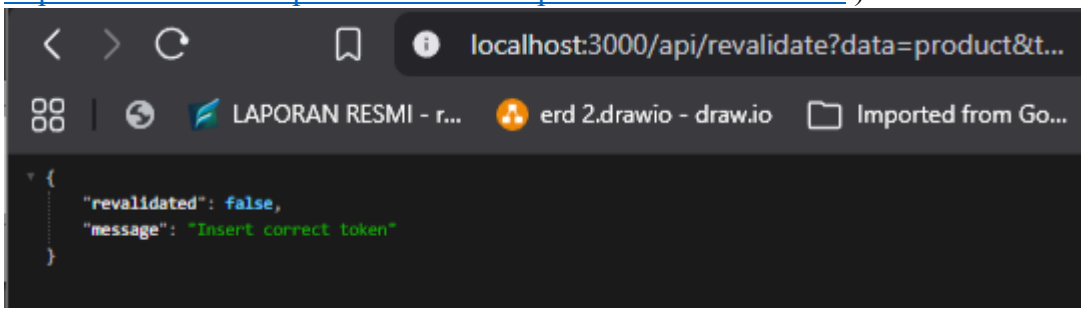
4. Tambahkan validasi token.

```
REVALIDATE_TOKEN=12345678
```

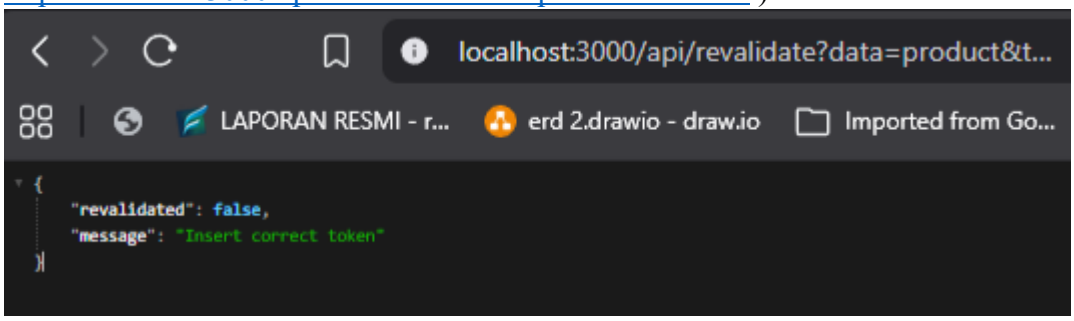
5. Uji dengan:
   - Token benar (menggunakan http://localhost:3000/api/revalidate?data=product&token=12345678)



   - Token salah (menggunakan http://localhost:3000/api/revalidate?data=product&token=123456)



   - Tanpa token (menggunakan http://localhost:3000/api/revalidate?data=product&token=)



PERTANYAAN ANALISIS

1. Mengapa ISR lebih fleksibel dibanding SSG?
   ISR (Incremental Static Regeneration) lebih fleksibel karena halaman yang sudah di-*build* tidak perlu di-*build ulang secara manual* untuk mendapatkan data terbaru. Pada praktikum kamu, dengan revalidate: 10, halaman otomatis memperbarui data setiap 10 detik setelah diakses, sehingga tetap cepat seperti SSG tetapi juga bisa mengikuti perubahan data di database tanpa harus menjalankan npm run build lagi.

2. Apa perbedaan revalidate waktu dan on-demand?
   Revalidate waktu (revalidate: 10) bekerja otomatis berdasarkan interval waktu tertentu, artinya data akan diperbarui setelah waktu tersebut ketika halaman di-*refresh*. Sedangkan on-demand revalidation bekerja secara manual melalui API (misalnya /api/revalidate), sehingga update hanya terjadi saat endpoint tersebut dipanggil, seperti yang kamu lakukan dengan parameter data=product.

3. Mengapa endpoint revalidation harus diamankan?
   Endpoint revalidation harus diamankan karena endpoint ini bisa memicu pembaruan data halaman secara langsung. Pada praktikum kamu, ditambahkan token agar tidak semua orang bisa mengakses URL tersebut. Tanpa pengamanan, siapa saja bisa memicu revalidasi dan berpotensi mengganggu performa atau memanipulasi perilaku aplikasi.

4. Apa risiko jika token tidak digunakan?
   Jika token tidak digunakan, endpoint revalidation bisa diakses oleh publik tanpa batas. Risiko yang terjadi adalah spam request ke endpoint, beban server meningkat, serta potensi penyalahgunaan seperti trigger revalidasi terus-menerus yang bisa menyebabkan performa aplikasi menurun atau tidak stabil.

5. Kapan ISR lebih cocok dibanding SSR?
   ISR lebih cocok digunakan ketika data tidak harus selalu real-time, tetapi tetap perlu diperbarui secara berkala, seperti pada kasus produk e-commerce di praktikum kamu. Dibanding SSR yang selalu render di server setiap request, ISR lebih efisien karena tetap menggunakan halaman statis namun bisa update otomatis atau manual saat diperlukan.